SISTEMAS OPERATIVOS (UADE) — TRABAJO PRÁCTICO OBLIGATORIO (TPO)

DOCUMENTO UNIFICADO DE PRESENTACIÓN Y DEFENSA (UNIDADES 1 A 6)

📋 Datos del Equipo (Consignas 1 y 2)

- **Materia:** Sistemas Operativos — Universidad Argentina de la Empresa (UADE)
- **Plataforma de Ejecución:** Red Hat Enterprise Linux (RHEL) — Consola Bash y C (`gcc`)
- **Integrantes del Grupo (4 miembros):**
    1. `[Nombre y Apellido — Legajo]`
    2. `[Nombre y Apellido — Legajo]`
    3. `[Nombre y Apellido — Legajo]`
    4. `[Nombre y Apellido — Legajo]`

📊 Resumen Ejecutivo de Cumplimiento de Consignas

Unidad	Ejercicio Práctico	Shell Scripting (Consigna 14)	Lenguaje C (Consigna 15)	Herramientas Consola Red Hat (Consigna 10)
Unidad 1	Diagnóstico de fallo en servicio mediante auditoría de *Syscalls*	✅ `sync_service.sh` y `audit_syscalls.sh`	—	`strace`, `grep`, `chmod`
Unidad 2	Condición de Carrera en hilos concurrentes vs. Semáforo *Mutex*	✅ `demostracion_tpo.sh`	✅ `carrera.c` y `seguro.c`	`gcc -lpthread`, `time`
Unidad 3	Asignación de Memoria Contigua (*First-Fit* vs *Best-Fit*)	✅ `ejecutar_tpo.sh`	✅ `simulador_memoria.c`	`gcc`, `time`
Unidad 4	Sistema de Archivos: Asignación Contigua vs. Nodos-i (*Inodes*)	✅ `ejecutar_tpo.sh`	✅ `simulador_filesystem.c`	`gcc`, `ls -li`, `stat`, `file`
Unidad 5	Subsistema de E/S: Escritura Sin Búfer vs. Con Búfer en Bloque	✅ `ejecutar_tpo.sh`	✅ `sin_bufer.c` y `con_bufer.c`	`gcc`, `time`, `strace -c`
Unidad 6	Seguridad: Firewall entre servidores, Usuarios/Shadow, ACL y Hash	✅ `ejecutar_tpo.sh` (Opciones 1 y 2)	—	`firewall-cmd`, `useradd`, `setfacl` / `getfacl`, `sha256sum`

🔷 UNIDAD 1: INTRODUCCIÓN A LOS SISTEMAS OPERATIVOS Y SYSCALLS

1.1 Problema Hipotético (Caso Empresarial)

En un servidor **Red Hat Enterprise Linux** de una entidad financiera, el servicio nocturno de conciliación (`sync_service.sh`) se detiene abruptamente arrojando un mensaje genérico (`[ERROR FATAL 101]`) sin especificar cuál es la falla. El administrador necesita descubrir la causa raíz sin modificar el programa, auditando las llamadas al sistema que realiza el proceso ante el Kernel.

1.2 Breve Marco Teórico a Exponer

- **Modo Usuario (Ring 3) vs. Modo Kernel (Ring 0):** Los programas de usuario no pueden acceder directo al disco por seguridad. Para abrir o leer un archivo deben pedirle el servicio al Kernel mediante una **Llamada al Sistema (*Syscall*)**, generando una interrupción por software (*trap*) que cambia el procesador a Modo Kernel.
- **Herramienta `strace`:** Intercepta y registra todas las llamadas al sistema (`openat` , `read` , `write` , `execve`) y el valor de retorno que devuelve el Kernel (como `-1 ENOENT` cuando un archivo no existe).

1.3 Prints del Código (`unidad_1/`)

📄 Archivo 1: `sync_service.sh` (Servicio que falla)

```
bash
#!/bin/bash
DATA_FILE="/etc/secure_app/transactions.dat"

echo "[INFO] Iniciando el servicio de conciliacion..."

if [ ! -r "$DATA_FILE" ]; then
   echo "[ERROR FATAL 101]: Fallo critico en el motor transaccional. Contacte a
soporte de IT." >&2
    exit 101
fi

cat "$DATA_FILE"
exit 0
```

📄 **Archivo 2:** `audit_syscalls.sh` **(Auditor con** `strace` **)**

```
bash
#!/bin/bash
chmod +x ./sync_service.sh

echo "============================================================"
echo " 1. INTERCEPTANDO SYSCALLS CON STRACE"
echo "============================================================"
# -f sigue procesos hijos, -e trace=file intercepta llamadas al sistema de archivos
strace -f -e trace=file -o auditoria_archivos.log ./sync_service.sh

echo ""
echo "============================================================"
echo " 2. DIAGNOSTICO: CAUSA RAIZ DETECTADA A NIVEL KERNEL"
echo "============================================================"
# Filtramos la llamada sobre el archivo que retornó código de error
grep "transactions.dat" auditoria_archivos.log

echo ""
echo "============================================================"
echo " 3. TABLA ESTADISTICA DE LLAMADAS AL SISTEMA (-c)"
echo "============================================================"
# -c genera el resumen de llamadas, errores y tiempo de procesador en modo núcleo
strace -c -e trace=execve,openat,read,write,close,newfstatat,faccessat2
./sync_service.sh
```

1.4 Salida en Consola Red Hat yExplicación del Resultado

```
text
============================================================
 2. DIAGNOSTICO: CAUSA RAIZ DETECTADA A NIVEL KERNEL
============================================================
faccessat2(AT_FDCWD, "/etc/secure_app/transactions.dat", R_OK, AT_EACCESS) = -1
ENOENT (No such file or directory)
```

> **Explicación para el profesor:** Con `strace` descubrimos que el Kernel devolvió `-1 ENOENT` porque el archivo `/etc/secure_app/transactions.dat` no existe en esa ruta.

`[INSERTAR CAPTURA DE PANTALLA DE LA CONSOLA RED HAT – UNIDAD 1]`

🔹 UNIDAD 2: PROCESOS, HILOS Y CONCURRENCIA (MUTEX)

2.1 Problema Hipotético (Caso Empresarial)

Un sistema bancario en **Red Hat** procesa depósitos simultáneos mediante **8 hilos de ejecución (*threads*)** que realizan `250.000` operaciones cada uno sobre una misma variable global (`contador_compartido`). Al finalizar el día, en lugar de registrar los **2.000.000** de depósitos esperados ($8 \times 250.000$8 × 250.000), el saldo final da un número menor y distinto en cada ejecución.

2.2 Breve Marco Teórico a Exponer

- **Condición de Carrera (*Race Condition*):** Ocurre cuando varios hilos leen y modifican una variable compartida al mismo tiempo sin sincronización; la operación `contador++` no es atómica en ensamblador (lee de RAM a registro, suma 1 y escribe en RAM), por lo que los hilos se pisan entre sí.
- **Sección Crítica y Exclusión Mutua (`pthread_mutex_t`):** La porción de código donde se modifica el recurso compartido es la **Sección Crítica**. Para evitar que dos hilos entren a la vez, se coloca un cerrojo **Mutex** (`pthread_mutex_lock` al entrar y `pthread_mutex_unlock` al salir).

2.3 Prints del Código (`unidad_2/`)

📄 Archivo 1: `carrera.c` (Sin sincronización — Con Condición de Carrera)

```
c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_HILOS 8
#define ITERACIONES 250000

long contador_compartido = 0;

void* depositar(void* arg) {
    for (int i = 0; i < ITERACIONES; i++) {
        // Sección crítica desprotegida (genera race condition)
        contador_compartido++;
    }
    return NULL;
}

int main() {
    pthread_t hilos[NUM_HILOS];

    for (int i = 0; i < NUM_HILOS; i++) {
        pthread_create(&hilos[i], NULL, depositar, NULL);
    }
    for (int i = 0; i < NUM_HILOS; i++) {
        pthread_join(hilos[i], NULL);
    }

    printf("[RACE CONDITION] Total esperado: %ld | Total obtenido: %ld\n",
           (long)NUM_HILOS * ITERACIONES, contador_compartido);
    return 0;
}
```

📄 Archivo 2: `seguro.c` (Solucionado con Exclusión Mutua `pthread_mutex`)

```
c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_HILOS 8
#define ITERACIONES 250000

long contador_compartido = 0;
pthread_mutex_t cerrojo;

void* depositar_seguro(void* arg) {
    for (int i = 0; i < ITERACIONES; i++) {
        pthread_mutex_lock(&cerrojo);   // Entrada a la sección crítica
        contador_compartido++;          // Sección crítica protegida
        pthread_mutex_unlock(&cerrojo); // Salida de la sección crítica
    }
    return NULL;
}

int main() {
    pthread_t hilos[NUM_HILOS];
    pthread_mutex_init(&cerrojo, NULL);

    for (int i = 0; i < NUM_HILOS; i++) {
        pthread_create(&hilos[i], NULL, depositar_seguro, NULL);
    }
    for (int i = 0; i < NUM_HILOS; i++) {
        pthread_join(hilos[i], NULL);
    }

    pthread_mutex_destroy(&cerrojo);
    printf("[MUTEX LOCK]     Total esperado: %ld | Total obtenido: %ld\n",
           (long)NUM_HILOS * ITERACIONES, contador_compartido);
    return 0;
}
```

📄 Archivo 3: `demostracion_tpo.sh` (Compilación y prueba con `time`)

```
bash
#!/bin/bash
echo "=== DEMOSTRACION CONDICION DE CARRERA - UNIDAD 2 ==="
gcc carrera.c -o carrera -lpthread
gcc seguro.c -o seguro -lpthread

echo -e "\n1. Ejecutando programa con condicion de carrera (sin sincronizacion):"
./carrera

echo -e "\n2. Ejecutando programa seguro con pthread_mutex (con exclusion mutua):"
./seguro

echo -e "\n3. Medición de tiempos de ejecucion:"
time ./carrera
time ./seguro
```

2.4 Salida en Consola Red Hat y Explicación del Resultado

```
text
1. Ejecutando programa con condicion de carrera (sin sincronizacion):
[RACE CONDITION] Total esperado: 2000000 | Total obtenido: 648912

2. Ejecutando programa seguro con pthread_mutex (con exclusion mutua):
[MUTEX LOCK]     Total esperado: 2000000 | Total obtenido: 2000000
```

> **Explicación para el profesor:** `carrera.c` pierde más de un millón de operaciones por la condición de carrera, pero es más rápido; `seguro.c` garantiza el resultado exacto (`2000000`) a cambio de una leve demora por bloquear y desbloquear el Mutex.

`[INSERTAR CAPTURA DE PANTALLA DE LA CONSOLA RED HAT – UNIDAD 2]`

🔹 UNIDAD 3: ADMINISTRACIÓN DE MEMORIA

3.1 Problema Hipotético (Caso Empresarial)

Un servidor **Red Hat** cuenta con **5 bloques libres de memoria RAM** separados (`100` , `500` , `200` , `300` y `600 KB` = **1700 KB libres en total**). Llegan **4 procesos** que piden memoria contigua (`212` , `417` , `112` y `426 KB` = **1167 KB**). Al usar el algoritmo **Primer Ajuste (*First-Fit*)**, el **Proceso 4 (426 KB) es rechazado** por falta de memoria aunque sobran **959 KB libres**.

3.2 Breve Marco Teórico a Exponer

- **Fragmentación Externa:** Existe suficiente memoria libre total para cargar un proceso, pero no se puede asignar porque está dividida en huecos pequeños no contiguos.
- **Primer Ajuste (*First-Fit*) vs. Mejor Ajuste (*Best-Fit*):**
    - *First-Fit (`F`):* Asigna el **primer hueco** donde entra el proceso. Es más rápido porque corta la búsqueda con `break` , pero desperdicia los huecos grandes al principio.
    - *Best-Fit (`B`):* Recorre todos los huecos y elige el **más ajustado**, logrando que entren los 4 procesos sin rechazos.

3.3 Prints del Código (`unidad_3/`)

📄 **Archivo 1:** `simulador_memoria.c`

```
c
#include <stdio.h>
#include <string.h>

#define CANT_HUECOS 5
#define CANT_PROCESOS 4

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Uso: %s [F|B] (F: First-Fit, B: Best-Fit)\n", argv[0]);
        return 1;
    }

    char modo = argv[1][0];
    int huecos[CANT_HUECOS] = {100, 500, 200, 300, 600};
    int procesos[CANT_PROCESOS] = {212, 417, 112, 426};
    int asignacion[CANT_PROCESOS] = {-1, -1, -1, -1};

    if (modo == 'F' || modo == 'f') {
        for (int i = 0; i < CANT_PROCESOS; i++) {
            for (int j = 0; j < CANT_HUECOS; j++) {
                if (huecos[j] >= procesos[i]) {
                    asignacion[i] = j;
                    huecos[j] -= procesos[i];
                    break; // First-Fit: corta en el primero que encuentra
                }
            }
        }
    } else {
        for (int i = 0; i < CANT_PROCESOS; i++) {
            int mejor_idx = -1;
            for (int j = 0; j < CANT_HUECOS; j++) {
                if (huecos[j] >= procesos[i]) {
                    if (mejor_idx == -1 || huecos[j] < huecos[mejor_idx]) {
                        mejor_idx = j; // Best-Fit: busca el hueco mas justo
                    }
                }
            }
            if (mejor_idx != -1) {
                asignacion[i] = mejor_idx;
                huecos[mejor_idx] -= procesos[i];
            }
```

```
        }
    }

    printf("\n[RESULTADO DE LA ASIGNACION (%c)]\n", modo);
    printf("Proceso\tTamano\tHueco Asignado\tEstado\n");
    for (int i = 0; i < CANT_PROCESOS; i++) {
        printf("P%d\t%d KB\t", i + 1, procesos[i]);
        if (asignacion[i] != -1)
            printf("Hueco %d\t\tASIGNADO\n", asignacion[i] + 1);
        else
            printf("Ninguno\t\tRECHAZADO (Sin espacio contiguo)\n");
    }
    return 0;
}
```

📄 **Archivo 2:** `ejecutar_tpo.sh`

```
bash
#!/bin/bash
gcc simulador_memoria.c -o simulador_memoria
./simulador_memoria F
./simulador_memoria B
time ./simulador_memoria F > /dev/null
time ./simulador_memoria B > /dev/null
```

3.4 Salida en Consola Red Hat y Explicación del Resultado

```
text
[RESULTADO DE LA ASIGNACION (F - FIRST-FIT)]
P1   212 KB   Hueco 2   ASIGNADO
P2   417 KB   Hueco 5   ASIGNADO
P3   112 KB   Hueco 2   ASIGNADO
P4   426 KB   Ninguno   RECHAZADO (Sin espacio contiguo) -> Sobran 959 KB libres!

[RESULTADO DE LA ASIGNACION (B - BEST-FIT)]
P1   212 KB   Hueco 4   ASIGNADO
P2   417 KB   Hueco 2   ASIGNADO
P3   112 KB   Hueco 3   ASIGNADO
P4   426 KB   Hueco 5   ASIGNADO -> 100% de procesos asignados!
```

`[INSERTAR CAPTURA DE PANTALLA DE LA CONSOLA RED HAT – UNIDAD 3]`

🔹 UNIDAD 4: SISTEMA DE ARCHIVOS (FILE SYSTEM)

4.1 Problema Hipotético (Caso Empresarial)

Llevamos el mismo escenario al almacenamiento en disco: un servidor tiene **5 huecos libres de bloques en el disco** (`100` , `500` , `200` , `300` , `600` bloques) tras borrar archivos viejos, y necesita guardar **4 archivos** (`212` , `417` , `112` , `426` bloques). Con **Asignación Contigua (`F`)**, el Archivo 4 no entra aunque sobran 959 bloques libres. Se soluciona implementando el **Modo `I` (Asignación con Inodos)** que usa Red Hat Linux (`ext4`).

4.2 Breve Marco Teórico a Exponer

- **Asignación Contigua vs. Asignación Indexada con Nodos-i (Inodos):**
    - La *Asignación Contigua* exige que todos los bloques de un archivo estén seguidos en el disco: sufre **fragmentación externa** y no deja que los archivos crezcan si el bloque de al lado está ocupado.
    - Los **Inodos (`ext4` en Linux)** son una "ficha índice" propia de cada archivo que guarda sus atributos (dueño, permisos, tamaño, fechas —**excepto el nombre**, que va en el directorio—) y **punteros a los bloques de datos dispersos** en cualquier parte del disco.

4.3 Prints del Código (`unidad_4/`)

📄 **Archivo 1:** `simulador_filesystem.c`

```
c
#include <stdio.h>

#define CANT_HUECOS 5
#define CANT_ARCHIVOS 4

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Uso: %s [F|B|I] (F: First-Fit Contiguo, B: Best-Fit Contiguo, I:
Inodos Linux)\n", argv[0]);
        return 1;
    }

    char modo = argv[1][0];
    int huecos[CANT_HUECOS] = {100, 500, 200, 300, 600};
    int archivos[CANT_ARCHIVOS] = {212, 417, 112, 426};
    int asignacion[CANT_ARCHIVOS] = {-1, -1, -1, -1};

    if (modo == 'F' || modo == 'f') {
        for (int i = 0; i < CANT_ARCHIVOS; i++) {
            for (int j = 0; j < CANT_HUECOS; j++) {
                if (huecos[j] >= archivos[i]) {
                    asignacion[i] = j;
                    huecos[j] -= archivos[i];
                    break;
                }
            }
        }
    } else if (modo == 'B' || modo == 'b') {
        for (int i = 0; i < CANT_ARCHIVOS; i++) {
            int mejor = -1;
            for (int j = 0; j < CANT_HUECOS; j++) {
                if (huecos[j] >= archivos[i] && (mejor == -1 || huecos[j] <
huecos[mejor]))
                    mejor = j;
            }
            if (mejor != -1) {
                asignacion[i] = mejor;
                huecos[mejor] -= archivos[i];
            }
        }
    } else if (modo == 'I' || modo == 'i') {
```

```
        // Modo Inodos: reparte los bloques del archivo en cualquier hueco libre
disponible
        for (int i = 0; i < CANT_ARCHIVOS; i++) {
            int falta_guardar = archivos[i];
            for (int j = 0; j < CANT_HUECOS && falta_guardar > 0; j++) {
                if (huecos[j] > 0) {
                    int usar = (huecos[j] >= falta_guardar) ? falta_guardar :
huecos[j];
                    huecos[j] -= usar;
                    falta_guardar -= usar;
                }
            }
            if (falta_guardar == 0) asignacion[i] = 99; // 99 = Guardado con Inodo
        }
    }

    printf("[RESULTADO AL GUARDAR LOS ARCHIVOS EN EL DISCO (MODO %c)]\n", modo);
    for (int i = 0; i < CANT_ARCHIVOS; i++) {
        printf("Archivo %d (%d bloques): ", i + 1, archivos[i]);
        if (asignacion[i] == 99) printf("GUARDADO (Inodo repartio los bloques en
huecos libres)\n");
        else if (asignacion[i] != -1) printf("GUARDADO en Hueco %d (Contiguo)\n",
asignacion[i] + 1);
        else printf("ERROR: NO ENTRA (Falta espacio contiguo!)\n");
    }
    return 0;
}
```

📄 Archivo 2: `ejecutar_tpo.sh`

```
bash
#!/bin/bash
gcc simulador_filesystem.c -o simulador_filesystem
./simulador_filesystem F
./simulador_filesystem B
./simulador_filesystem I

echo "--- VIENDO UN INODO REAL Y NUMERO MAGICO EN LINUX ---"
ls -li simulador_filesystem.c
stat simulador_filesystem.c
file simulador_filesystem.c
file simulador_filesystem
```

4.4 Salida en Consola Red Hat y Explicación del Resultado

```
text
[RESULTADO AL GUARDAR LOS ARCHIVOS EN EL DISCO (MODO F)]
Archivo 4 (426 bloques): ERROR: NO ENTRA (Falta espacio contiguo!)

[RESULTADO AL GUARDAR LOS ARCHIVOS EN EL DISCO (MODO I)]
Archivo 1 (212 bloques): GUARDADO (Inodo repartio los bloques en huecos libres)
Archivo 2 (417 bloques): GUARDADO (Inodo repartio los bloques en huecos libres)
Archivo 3 (112 bloques): GUARDADO (Inodo repartio los bloques en huecos libres)
Archivo 4 (426 bloques): GUARDADO (Inodo repartio los bloques en huecos libres)
```

`[INSERTAR CAPTURA DE PANTALLA DE LA CONSOLA RED HAT — UNIDAD 4]`

🔹 UNIDAD 5: SUBSISTEMA DE ENTRADA / SALIDA (E/S)

5.1 Problema Hipotético (Caso Empresarial)

Un servidor **Red Hat** registra un consumo excesivo de CPU en Modo Kernel (`sys`) y gran lentitud al guardar un archivo de registro de apenas **100.000 caracteres**. Al auditarlo con `strace -c`, se detecta que el programa (`sin_bufer.c`) escribe de a **1 letra por llamada al sistema** `write()` (100.000 viajes al Kernel). Se soluciona con `con_bufer.c`, que agrupa **4.000 letras en un Búfer en memoria** antes de escribir.

5.2 Breve Marco Teórico a Exponer

- **Las 4 Capas del Software de E/S:** 1) Espacio de Usuario, 2) Software de E/S Independiente del Dispositivo, 3) Drivers de Dispositivos y 4) Manejadores de Interrupciones.
- **Importancia del Uso de Búfer (*Buffering*):** Pertenece a la capa independiente del dispositivo y al espacio de usuario. Permite **desacoplar velocidades** y **amortiguar el costo de las llamadas al sistema**: en lugar de cruzar de Modo Usuario a Modo Kernel 100.000 veces (1 byte por vez), juntar 4.000 bytes en un búfer reduce las llamadas al sistema a **solo 25 viajes** ($25 \times 4000 = 100.000$25 × 4000 = 100.000).

5.3 Prints del Código (`unidad_5/`)

📄 Archivo 1: `sin_bufer.c` (Lento — 100.000 llamadas a `write`)

```
c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("salida_lenta.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) return 1;

    printf("[SIN BUFER]: Escribiendo 100.000 letras de a 1 byte por vez...\n");
    for (int i = 0; i < 100000; i++) {
        write(fd, "A", 1); // 1 llamada al sistema por cada letra
    }

    close(fd);
    printf("[SIN BUFER]: Terminado (100.000 llamadas a write).\n");
    return 0;
}
```

📄 Archivo 2: `con_bufer.c` (Rápido — Solo 25 llamadas a `write`)

```
c
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("salida_rapida.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) return 1;

    char bufer[4000];
    memset(bufer, 'A', 4000); // Llenamos el bufer con 4000 letras 'A'

    printf("[CON BUFER]: Escribiendo 100.000 letras en bloques de 4000 bytes...\n");
    for (int i = 0; i < 25; i++) {
        write(fd, bufer, 4000); // 1 sola llamada escribe 4000 letras juntas
    }

    close(fd);
    printf("[CON BUFER]: Terminado (Solo 25 llamadas a write!).\n");
    return 0;
}
```

📄 Archivo 3: `ejecutar_tpo.sh`

```
bash
#!/bin/bash
gcc sin_bufer.c -o sin_bufer && gcc con_bufer.c -o con_bufer
time ./sin_bufer
time ./con_bufer

echo "-> Cantidad de syscalls SIN BUFER:"
strace -c -e trace=write ./sin_bufer > /dev/null
echo "-> Cantidad de syscalls CON BUFER:"
strace -c -e trace=write ./con_bufer > /dev/null
rm -f salida_lenta.txt salida_rapida.txt
```

5.4 Salida en Consola Red Hat y Explicación del Resultado

```
text
-> Cantidad de syscalls SIN BUFER:
% time     seconds  usecs/call     calls    errors syscall
100.00    0.184000           1    100000           write

-> Cantidad de syscalls CON BUFER:
% time     seconds  usecs/call     calls    errors syscall
100.00    0.000050           2        25           write
```

> **Explicación para el profesor:** Ambos programas escriben el mismo archivo de 100.000 letras, pero `con_bufer` hace **25 llamadas al sistema en lugar de 100.000**, eliminando la sobrecarga del procesador en Modo Kernel.

`[INSERTAR CAPTURA DE PANTALLA DE LA CONSOLA RED HAT — UNIDAD 5]`

🔹 UNIDAD 6: SEGURIDAD EN SISTEMAS OPERATIVOS

6.1 Problema Hipotético (Caso Empresarial)

En una infraestructura de servidores **Red Hat** (`workstation` , `servera` y `serverb`), la empresa necesita aplicar **Defensa en Profundidad** para:

1. Verificar la conectividad entre servidores y configurar el **Firewall (`firewall-cmd`)** para **bloquear** el puerto `8080/tcp` en `servera` y **habilitarlo** en `serverb` .
2. Crear un usuario de auditoría (`auditor`), verificar que su contraseña se almacene con **Hash + Salt en `/etc/shadow`** , y otorgarle acceso exclusivo a un archivo confidencial mediante una **Lista de Control de Acceso (`setfacl` / `getfacl`)**.
3. Proteger el script `pagos_empresa.sh` con un **Comprobador de Integridad (`sha256sum -c`)** que detecte y bloquee su ejecución si un virus lo modifica.

6.2 Breve Marco Teórico a Exponer

- **Los 4 Objetivos de la Seguridad:** 1) **Confidencialidad** (protegida con permisos y ACLs), 2) **Integridad** (protegida con funciones de una vía / Hash `sha256sum`), 3) **Disponibilidad** y 4) **Exclusión de usuarios externos** (protegida con el Firewall).

- **Funciones de Una Vía (Hash) y *Salt* en UNIX:** Linux guarda en `/etc/shadow` el Hash de la contraseña concatenada con un texto aleatorio (**Salt**) para inutilizar los ataques de diccionario precalculado.
- **Listas de Control de Acceso (ACL):** Guardan la matriz de protección por columna junto al archivo (`setfacl` / `getfacl`), permitiendo dar permiso a un usuario puntual sin abrir el archivo al resto.

6.3 Prints del Código (`unidad_6/` — Incluye las 2 Opciones)

📄 Opción 1: `opcion1_servers_remotos.sh` (Con `servera` y `serverb`)

```
bash
#!/bin/bash
PUERTO=8080

echo "--- 1. VERIFICANDO CONECTIVIDAD DE RED CON 'servera' Y 'serverb' ---"
ping -c 2 servera
ping -c 2 serverb

echo "--- 2. FIREWALL: BLOQUEAR PUERTO EN 'servera' Y ABRIR EN 'serverb' ---"
ssh student@servera "sudo firewall-cmd --remove-port=${PUERTO}/tcp >/dev/null 2>&1;
sudo firewall-cmd --list-ports"
ssh student@serverb "sudo firewall-cmd --add-port=${PUERTO}/tcp >/dev/null 2>&1;
nohup python3 -m http.server $PUERTO >/dev/null 2>&1 &"
sleep 1

echo "1) Intentando conectar a servera:$PUERTO (Puerto BLOQUEADO por Firewall):"
if curl -s --connect-timeout 2 http://servera:$PUERTO >/dev/null; then
    echo "   [OK]: Conectado a servera."
else
    echo "   [BLOQUEADO]: El Firewall de 'servera' rechazo la conexion al puerto
$PUERTO!"
fi

echo "2) Intentando conectar a serverb:$PUERTO (Puerto HABILITADO en Firewall):"
if curl -s --connect-timeout 2 http://serverb:$PUERTO >/dev/null; then
    echo "   [PERMITIDO]: Conexion exitosa con 'serverb' en el puerto $PUERTO!"
fi

echo "--- 3. CREACION DE USUARIO AUTORIZADO ('auditor') Y HASH+SALT EN /etc/shadow -
--"
sudo useradd -m auditor 2>/dev/null
echo "auditor:RedHat2026" | sudo chpasswd
sudo grep "^auditor:" /etc/shadow

echo "--- 4. CONTROL DE ACCESO (ACL): PERMISO EXCLUSIVO AL USUARIO 'auditor' ---"
ARCHIVO="/tmp/reporte_confidencial.txt"
echo "DATOS SECRETOS DEL SERVIDOR RED HAT" > $ARCHIVO
chmod 600 $ARCHIVO
setfacl -m u:auditor:r-- $ARCHIVO
getfacl $ARCHIVO
sudo su - auditor -c "cat $ARCHIVO"
```

📄 Opción 2: `opcion2_local_completo.sh` (En un solo servidor + Comprobador de Integridad Hash)

```
bash
#!/bin/bash
echo "--- 1. PUERTOS EN ESCUCHA (ss -tuln) Y REGLAS DE FIREWALL (firewall-cmd) ---"
ss -tuln | head -n 8
sudo firewall-cmd --add-port=8080/tcp
sudo firewall-cmd --remove-port=8080/tcp

echo "--- 2. CREAR USUARIO ('auditor') Y VER SU HASH + SALT EN /etc/shadow ---"
sudo useradd -m auditor 2>/dev/null
echo "auditor:RedHat2026" | sudo chpasswd
sudo grep "^auditor:" /etc/shadow

echo "--- 3. LISTA DE CONTROL DE ACCESO (ACL) PARA EL USUARIO AUTORIZADO ---"
ARCHIVO="/tmp/reporte_confidencial.txt"
echo "DATOS SECRETOS DEL SERVIDOR RED HAT" > $ARCHIVO
chmod 600 $ARCHIVO
setfacl -m u:auditor:r-- $ARCHIVO
getfacl $ARCHIVO
rm -f $ARCHIVO

echo "--- 4. COMPROBADOR DE INTEGRIDAD ANTI-VIRUS (HASH SHA-256 EN 'sumacomp.txt') -
--"
sha256sum pagos_empresa.sh > sumacomp.txt
sha256sum -c sumacomp.txt && ./pagos_empresa.sh

echo 'echo "[VIRUS]: Robando contrasenas..."' >> pagos_empresa.sh
if sha256sum -c sumacomp.txt; then
    ./pagos_empresa.sh
else
    echo "[BLOQUEADO]: PELIGRO! El Hash SHA-256 no coincide. Ejecucion cancelada!"
fi
sed -i '/VIRUS/d' pagos_empresa.sh
rm -f sumacomp.txt
```

6.4 Salida en Consola Red Hat y Explicación del Resultado

```
text
1) Intentando conectar a servera:8080 (Puerto BLOQUEADO por Firewall):
   [BLOQUEADO]: El Firewall de 'servera' rechazo la conexion al puerto 8080!
2) Intentando conectar a serverb:8080 (Puerto HABILITADO en Firewall):
   [PERMITIDO]: Conexion exitosa con 'serverb' en el puerto 8080!

-> Entrada real en /etc/shadow:
auditor:$6$xyzSalt$92a8f...:19651:0:99999:7:::

-> Verificacion de Integridad SHA-256 tras inyeccion del virus:
pagos_empresa.sh: La suma no coincide
[BLOQUEADO]: PELIGRO! El Hash SHA-256 no coincide. Ejecucion cancelada!
```

`[INSERTAR CAPTURA DE PANTALLA DE LA CONSOLA RED HAT — UNIDAD 6]`